



Politécnico
Internacional

ESTRUCTURA DE DATOS



**“El problema real no es si las maquinas piensan,
sino si lo hacen los hombres.”**

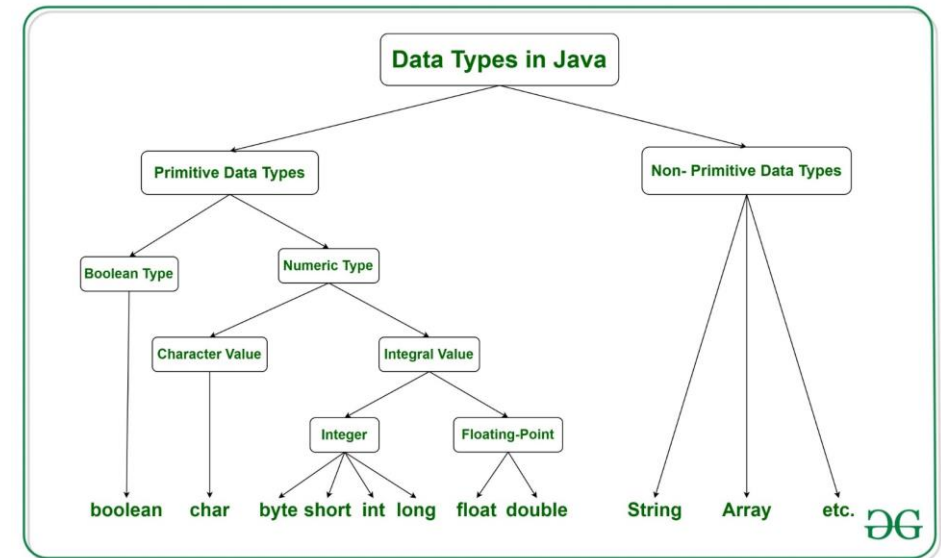
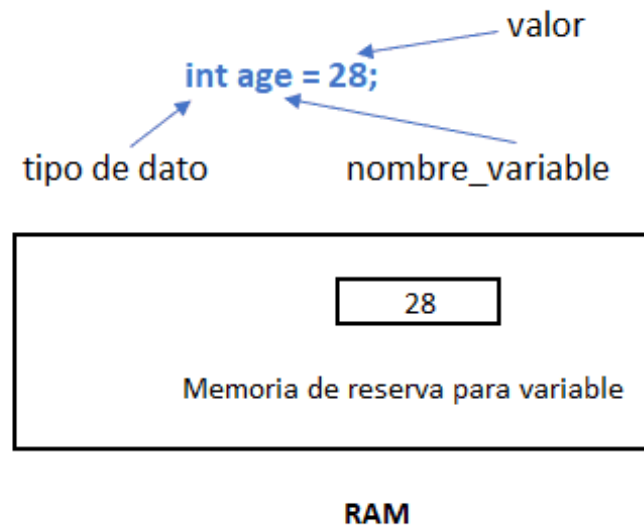
B. F – SKINNER

AGENDA

- Conceptos Java
- Innova Colombia
- Conclusiones

Retroalimentación

- ¿Qué es Java?
- Características – Seguridad
- POO
- Sintaxis Básica
- Historia
- Tipado – Estático & Dinámico
- Tipos de datos.
- Operadores



¿CÓMO DECLARA VARIABLES?

```
float simpleInterest; //Declarando variable float  
int time = 10, speed = 20; //Declarando e Inicializando la variable integer  
char var = 'h'; // Declarando e Inicializando la variable character
```

TIPOS DE VARIABLES

JAVA + TIPOS:

- *VARIABLES LOCALES*
- *VARIABLES DE INSTANCIA*
- *VARIABLES ESTÁTICAS*



Una variable definida dentro de un bloque, método o constructor se llama variable local.

Estas variables se crean cuando el bloque ingresado o método se llama y destruye después de salir del bloque o cuando la llamada regresa del método.

El alcance de estas variables solo existe dentro del bloque en el que se declara la variable, es decir, podemos acceder a estas variables solo dentro de ese bloque.

```
public class StudentDetails
{
    public void StudentAge()
    {    //variable local age
        int age = 0;
        age = age + 5;
        System.out.println("La edad del estudiante es : " + age);
    }

    public static void main(String args[])
    {
        StudentDetails obj = new StudentDetails();
        obj.StudentAge();
    }
}
```

En el programa anterior, la variable age (edad) es una variable local para el método `[java]StudentAge()[/java]`.

Si usamos la variable age fuera del método `[java]StudentAge()[/java]`, el compilador producirá un error como se muestra a continuación en el programa

```
public class StudentDetails
{
    public void StudentAge()
    {    //variable local age
        int age = 0;
        age = age + 5;
        System.out.println("La edad del estudiante es : " + age);
    }

    public static void main(String args[])
    {
        StudentDetails obj = new StudentDetails();
        obj.StudentAge();
    }
}
```

Las variables de instancia son variables no estáticas y se declaran en una clase fuera de cualquier método, constructor o bloque.

Como las variables de instancia se declaran en una clase, estas variables se crean cuando un objeto de la clase se crea y se destruye cuando se destruye el objeto.

A diferencia de las variables locales, podemos usar especificadores de acceso para variables de instancia. Si no especificamos ningún especificador de acceso, se utilizará el especificador de acceso predeterminado.

```
import java.io.*;
class Points
{
    //Estas variables son variables de instancia.
    //Estas variables están en una clase y no están dentro de ninguna función/método
    int engPoints;
    int mathsPoints;
    int phyPoints;
}

class PointsDemo
{
    public static void main(String args[])
    {
        //primer objeto
        Points obj1 = new Points();
        obj1.engPoints = 50;
        obj1.mathsPoints = 80;
        obj1.phyPoints = 90;

        //segundo objeto
        Points obj2 = new Points();
        obj2.engPoints = 80;
        obj2.mathsPoints = 60;
        obj2.phyPoints = 85;

        //mostrando puntos para el primer objeto
        System.out.println("Puntos para el primer objeto:");
        System.out.println(obj1.engPoints);
        System.out.println(obj1.mathsPoints);
        System.out.println(obj1.phyPoints);

        //mostrando puntos para el segundo objeto
        System.out.println("Puntos para el segundo objeto:");
        System.out.println(obj2.engPoints);
        System.out.println(obj2.mathsPoints);
        System.out.println(obj2.phyPoints);
    }
}
```



```
import java.io.*;
class Points
{
    //Estas variables son variables de instancia.
    //Estas variables están en una clase y no están dentro de ninguna función/método
    int engPoints;
    int mathsPoints;
    int phyPoints;
}

class PointsDemo
{
    public static void main(String args[])
    {
        //primer objeto
        Points obj1 = new Points();
        obj1.engPoints = 50;
        obj1.mathsPoints = 80;
        obj1.phyPoints = 90;

        //segundo objeto
        Points obj2 = new Points();
        obj2.engPoints = 80;
        obj2.mathsPoints = 60;
        obj2.phyPoints = 85;

        //mostrando puntos para el primer objeto
        System.out.println("Puntos para el primer objeto:");
        System.out.println(obj1.engPoints);
        System.out.println(obj1.mathsPoints);
        System.out.println(obj1.phyPoints);

        //mostrando puntos para el segundo objeto
        System.out.println("Puntos para el segundo objeto:");
        System.out.println(obj2.engPoints);
        System.out.println(obj2.mathsPoints);
        System.out.println(obj2.phyPoints);
    }
}
```



Las variables estáticas también se conocen como variables de clase.

Estas variables se declaran de forma similar a las variables de instancia, la diferencia es que las variables estáticas se declaran utilizando la palabra clave `[java]static[/java]` dentro de una clase fuera de cualquier constructor o bloque de métodos.

A diferencia de las variables de instancia, solo podemos tener una copia de una variable estática por clase, independientemente de cuántos objetos creemos.

Las variables estáticas se crean al inicio de la ejecución del programa y se destruyen automáticamente cuando finaliza la ejecución.

```
import java.io.*;
class Emp {

    // salario como variable estatica
    public static double salary;
    public static String name = "Alex";
}

public class EmpDemo
{
    public static void main(String args[]) {

        //acceder a la variable estatica sin objeto
        Emp.salary = 1000;
        System.out.println(Emp.name + " tiene un salario promedio de: " + Emp.salary);
    }
}
```

VARIABLES CONSTANTES

Las variables creadas para almacenar valores fijos de esta manera se conocen como “constantes”, y es convencional nombrar constantes con todos los caracteres en mayúsculas, para distinguirlas de las variables regulares. Los programas que intentan cambiar un valor constante no se compilarán, y el compilador javac generará un mensaje de error.

```
class Constants
{
    public static void main ( String[] args ) {
        //inicializamos tres constantes enteras
        final int CONSTANTE1 = 6 ;
        final int CONSTANTE2 = 1 ;
        final int CONSTANTE3 = 3 ;

        //declaramos variables regulares del tipo int
        int td,pat,fg,total;

        //Inicializamos las variables regulares
        td = 4*CONSTANTE1;
        pat = 3*CONSTANTE2;
        fg = 2*CONSTANTE3;

        total = (td+pat+fg) ;

        System.out.println("Resultado: " + total);
    }
}
```



Politécnico
Internacional



Java™

25 Years

LECTURA DE DATOS



*Al escribir un programa
debemos realizar lectura de los
datos ingresados por teclado.*



```
6 package cide_entrada_datos_buffered_reader;
7 import java.io.BufferedReader;
8 import java.io.IOException;
9 import java.io.InputStreamReader;
10 /**
11  *
12  * @author Harol
13  */
14 // CIDE TIPOS DE DATOS
15 // FUNDAMENTOS DE PROGRAMACION
16 public class CIDE_ENTRADA_DATOS_BUFFERED_READER {
17
18     /**
19      * @param args the command line arguments
20      */
21     public static void main(String[] args) throws IOException {
22         // TODO code application logic here
23         //Ingrese datos usando BufferedReader
24         BufferedReader reader =
25             new BufferedReader(new InputStreamReader(System.in));
26         // Leyendo datos usando readLine
27         String name = reader.readLine();
28
29         // Imprimir la línea de lectura
30         System.out.println(name);
31     }
32
33 }
```

CLASE BUFFERED READER



Este es el método clásico de Java para leer datos de entrada, introducido en JDK 1.0. Este método se usa envolviendo `System.in` (flujo de entrada estándar) en un `InputStreamReader` que está envuelto en un `BufferedReader`, podemos leer la entrada del usuario en la línea de comando



```
package cide_entrada_datos_clase_escaner;
import java.util.Scanner;
/**
 *
 * @author Harol
 */
// CIDE ENTRADA DE DATOS
// FUNDAMENTOS DE PROGRAMACION
public class CIDE_ENTRADA_DATOS_CLASE_ESCANER {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
        // Usando Scanner para obtener información del usuario
        Scanner in = new Scanner(System.in);
        String s = in.nextLine();
        System.out.println("Usted ingresó la cadena: "+s);
        int a = in.nextInt();
        System.out.println("Usted ingresó un entero: "+a);
        float b = in.nextFloat();
        System.out.println("Usted ingresó un float: "+b);
    }
}
```

CLASE ESCANER



Este es probablemente el método más preferido para tomar datos de entrada. El objetivo principal de la clase Scanner es analizar los tipos primitivos y las cadenas con expresiones regulares, sin embargo, también se puede usar para leer las entradas del usuario en la línea de comandos.



```
6 package cide_entrada_datos_joption_pane;
7 import javax.swing.*.*;
8 /**
9  *
10  * @author Harol
11  */
12 public class CIDE_ENTRADA_DATOS_JOPTION_PANE {
13
14     /**
15      * @param args the command line arguments
16      */
17     public static void main(String[] args) {
18         // TODO code application logic here
19         JOptionPane.showMessageDialog(null, "Programa que muestra los primeros 20 Numeros pares");
20         int i=0, ci;
21         for (int c=0; c<=38; c++){
22             i = i + 2;
23             c = i;
24             JOptionPane.showMessageDialog(null, c);
25         }
26         JOptionPane.showMessageDialog(null, "CIDE Fundamentos Programacion Harol Torres");
27     }
28 }
```

JOPTION_PANE

Opción diferente a utilizar la consola. Muestra un cuadro de dialogo para la interacción entre el usuario y el programa desarrollado.

Mensaje



Programa que muestra los primeros 20 Numeros pares

Aceptar



La toma de decisiones en la programación es similar a la toma de decisiones en la vida real.

En la programación también enfrentamos algunas situaciones en las que queremos que se ejecute cierto bloque de código cuando se cumple alguna condición.

Un lenguaje de programación utiliza instrucciones de control para controlar el flujo de ejecución del programa en función de ciertas condiciones. Estos se utilizan para hacer que el flujo de ejecución avance y se ramifique en función de los cambios en el estado de un programa.

ESTRUCTURAS CONDICIONALES JAVA

```
void compileFile(final SyntaxNode n) throws CodeException {
    for (Iterator<SyntaxNode> ite = n.getChildren().iterator(); ite.hasNext(); ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackageRule()) {
            // Handle package rules
        } else if (rule.isImportRule()) {
            // Handle import rules
        } else {
            // Handle other rules
        }
    }
}
```

ESTRUCTURAS CONDICIONALES JAVA

```
void createFile(final SyntaxNode n) throws CodeException {
    for (Iterator<SyntaxNode> ite = n.getChildren().iterator(); ite.hasNext(); ite = ite.next()) {
        final SyntaxNode child = (SyntaxNode) ite.next();
        final Rule rule = child.getRule();
        if (rule.isPackage() || rule.isImport()) {
            // TODO handle static and *
            final SyntaxNode n = child.getChildByRule(RULE_IMPORT);
            final String fullName = child.getChildByRule(RULE_PACKAGE).getChars();
            final String[] parts = fullName.split("\\.");
        } else {
            // ...
        }
    }
}
```



Declaraciones de selección de Java:

- *if*
- *if-else*
- *nested-if*
- *if-else-if*
- *switch-case*
- *jump – break, continue, return*

IF CONDICIONALES JAVA

```
if (condición)
{
    // Declaraciones para ejecutar si
    // la condición es verdadera
}
```



La sentencia if es la declaración de toma de decisiones más simple.

Se usa para decidir si una determinada declaración o bloque de enunciados se ejecutará o no; es decir, si una determinada condición es verdadera (true), se ejecutará un bloque de enunciado y, de ser falsa (false), no.

IF CONDICIONALES JAVA

```
if (condición)
{
    // Declaraciones para ejecutar si
    // la condición es verdadera
}
```



La sentencia if es la declaración de toma de decisiones más simple.

Se usa para decidir si una determinada declaración o bloque de enunciados se ejecutará o no; es decir, si una determinada condición es verdadera (true), se ejecutará un bloque de enunciado y, de ser falsa (false), no.

IF - ELSE CONDICIONALES JAVA

```
if (condición)
{
    // Ejecuta este bloque si
    // la condición es verdadera
}
else
{
    // Ejecuta este bloque si
    // la condición es falsa
}
```



La declaración *if* solo nos dice que, si una condición es verdadera ejecutará un bloque de instrucciones y si la condición es falsa, no lo hará. Pero, ¿y si queremos hacer otra cosa cuando la condición sea falsa? Aquí viene la declaración *else*. Podemos usar la instrucción *else* con la instrucción *if* para ejecutar un bloque de código cuando la condición es falsa.

IF - ANIDADO

CONDICIONALES JAVA

```
if (condicion1)
{
    // Ejecuta cuando condicion1 es verdadero
    if (condition2)
    {
        // Ejecuta cuando condicion2 es verdadero
    }
}
```



Una *if* anidado (*nested-if*) es una declaración *if* que se deriva de otro *if* o *else*. Las declaraciones *if* anidadas significan una instrucción *if* dentro de una declaración *if*. Sí, Java nos permite anidar las declaraciones *if* con *if*, es decir, podemos colocar una instrucción *if* dentro de otra instrucción *if*.

IF - ANIDADO

CONDICIONALES JAVA

```
if (condicion1)
{
    // Ejecuta cuando condicion1 es verdadero
    if (condition2)
    {
        // Ejecuta cuando condicion2 es verdadero
    }
}
```



Una *if* anidado (*nested-if*) es una declaración *if* que se deriva de otro *if* o *else*. Las declaraciones *if* anidadas significan una instrucción *if* dentro de una declaración *if*. Sí, Java nos permite anidar las declaraciones *if* con *if*, es decir, podemos colocar una instrucción *if* dentro de otra instrucción *if*.



ESCALERA IF - ELSE CONDICIONALES JAVA

```
if (condición)
    declaración;
else if (condición)
    declaración;
.
.
    declaración
else ;
```



Un usuario puede decidir entre múltiples opciones. Las sentencias *if* se ejecutan desde arriba hacia abajo. Tan pronto como una de las condiciones que controlan el *if* sea verdadera, se ejecuta la instrucción asociada con ese *if*, y el resto de la escalera se pasa por alto. Si ninguna de las condiciones es verdadera, se ejecutará la sentencia final *else*.



SWITCH CASE CONDICIONALES JAVA

```
switch (expresión)
{
    case valor1:
        declaracion1;
        break;
    case value2:
        declaracion2;
        break;
    .
    .
    case valorN:
        declaracionN;
        break;
    default:
        declaracionDefault;
}
```



instrucción switch es una declaración de bifurcación de múltiples vías (selección múltiple). Proporciona una forma sencilla de enviar la ejecución a diferentes partes del código en función del valor de la expresión.



Politécnico
Internacional



JUMP CONDICIONALES JAVA



Java admite tres declaraciones de salto: break, continue y return. Estas tres declaraciones transfieren el control a otra parte del programa.

BREAK CONDICIONALES JAVA

```
// Programa Java para ilustrar usando
// break para salir del bucle
class BreakLoopDemo
{
    public static void main(String args[])
    {
        // Inicialmente, el bucle está configurado para ejecutarse desde 0-9
        for (int i = 0; i < 10; i++)
        {
            // termina el bucle cuando llega a 5.
            if (i == 5)
                break;

            System.out.println("i: " + i);
        }
        System.out.println("Bucle completo.");
    }
}
```



En Java, break se utiliza principalmente para:

Terminar una secuencia en una instrucción switch (discutida arriba).

*Para salir de un bucle
Como una forma “civilizada” de goto.*

Usar break para salir de un bucle

Utilizando el break, podemos forzar la terminación inmediata de un bucle, evitando la expresión condicional y cualquier código restante en el cuerpo del bucle.



```
// Programa Java para ilustrar usando
// continue en una declaración if
class ContinueDemo
{
    public static void main(String args[])
    {
        for (int i = 0; i < 10; i++)
        {
            // Si el número es par
            // omitir y continuar
            if (i%2 == 0)
                continue;

            // Si el número es impar, imprímalo
            System.out.print(i + " ");

        }
    }
}
```

CONTINUE CONDICIONALES JAVA



es útil forzar una iteración temprana de un bucle. Es decir, es posible que desee continuar ejecutando el bucle, pero deje de procesar el resto del código (en su cuerpo) para esta iteración en particular. Esto es, en efecto, un **goto** pasando del cuerpo del bucle, al final del bucle. La instrucción **continue** realiza tal acción.

RETURN CONDICIONALES JAVA

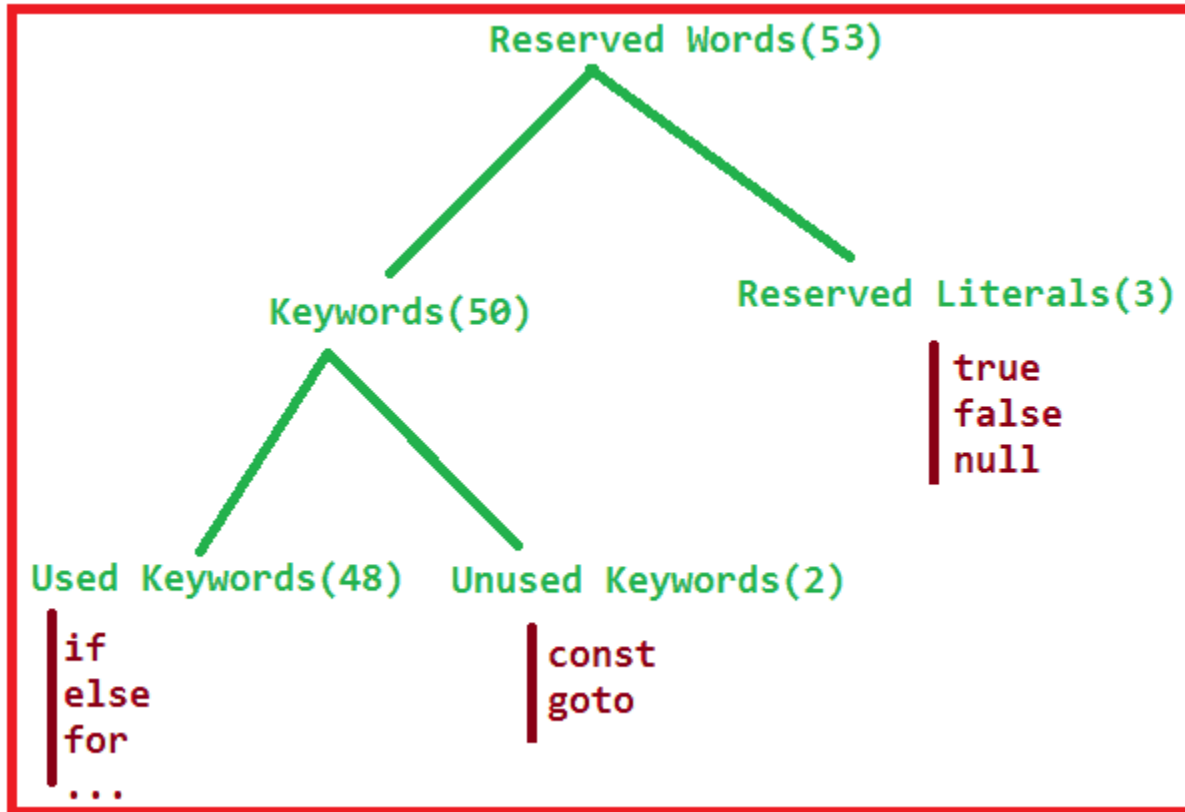


La declaración **RETURN** se usa para regresar explícitamente de un método. Es decir, hace que un control de programa se transfiera nuevamente a quién llama del método.

```
// Programa Java para ilustrar usando return
class Return
{
    public static void main(String args[])
    {
        boolean t = true;
        System.out.println("Antes de return.");

        if (t)
            return;

        // El compilador eludirá todas las declaraciones
        // después de return
        System.out.println("Esto no se ejecutará.");
    }
}
```



PALABRAS RESERVADAS EN JAVA

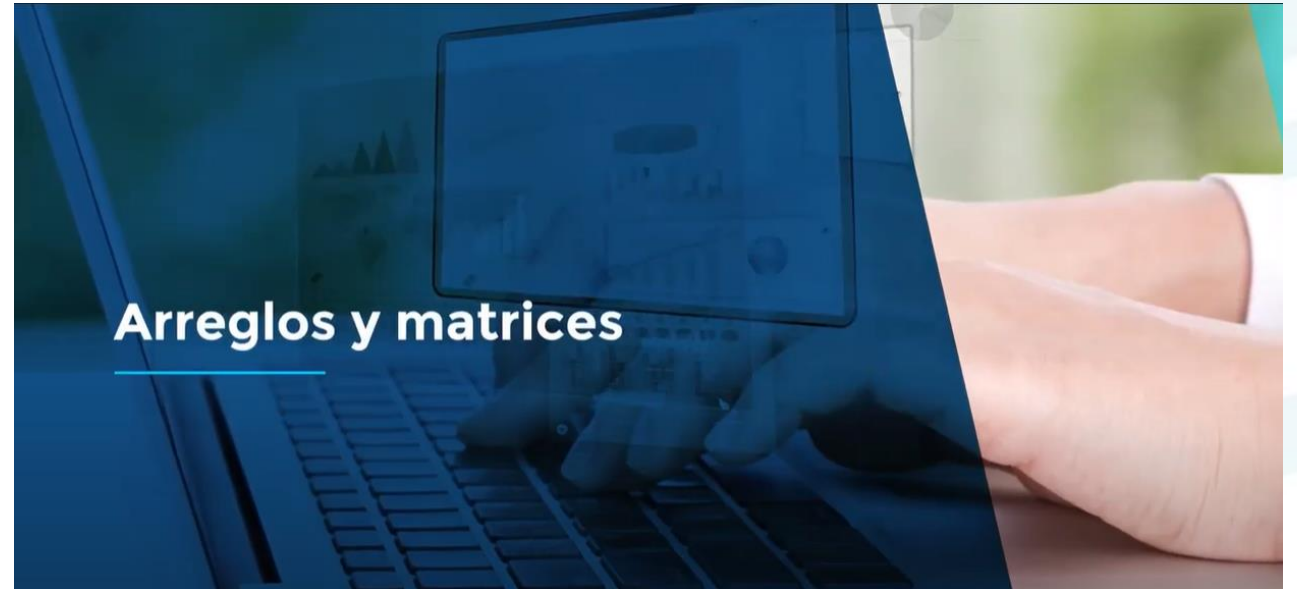
Las palabras clave definen funcionalidades y las literales definen un valor.

PALABRAS RESERVADAS EN JAVA

Tabla de Palabras Claves Reservadas en Java

abstract	assert	boolean	break	byte	case	catch	char	class	const	continue
default	do	double	else	enum	extends	false	final	finally	float	for
goto	if	implements	import	instanceof	int	interface	long	native	new	null
package	private	protected	public	return	short	static	strictfp	String	super	switch
synchronized	this	throw	throws	transient	true	try	void	volatile	while	

ARREGLOS Y MATRICES



<https://www.youtube.com/watch?v=W8QwbhynEmc&t=267s>



Politécnico
Internacional



AVANCES **INNOVA** COLOMBIA

¿preguntas?

RETO 1

CONCLUSIONES